

EZcs题解

出题思路

父子进程间通过一个特定的自定义协议去进行通信，而漏洞点主要体现在了协议中，并且关闭了0, 1, 2这三个文件描述符，因此客户端只能通过约定的通信方式来获得服务端的响应信息，正常来说，其实还会考察gdb同时调试两个进程，但是似乎被AI调试单个进程盲打通关了（不是。

漏洞利用链

首先，肯定要逆向还原程序的大致原貌，但是善良的出题人留了符号，没有全部strip掉。

泄露

难点主要就在于弄清如何解析的包，在弄清楚之后，get函数里并没有去检查index，仅仅检查了对应的index下是否有content，而仔细看会发现，在service启动的时候会创建一个0xa0的空间进行用户密码表的存储，当用户注册的时候，会malloc 0x150空间给用户，而删除的时候，只是清空了size和char *content，但是这个表肯定还是在对应的堆空间中，

```

pwndbg> tele 0x63668b760360 30
00:0000 0x63668b760360 ← 0
01:0008 0x63668b760368 ← 0x161
02:0010 rdx 0x63668b760370 → 0x63668b7604d0 ← 0x79686c /* 'lhy' */
03:0018 0x63668b760378 ← 3
04:0020 0x63668b760380 ← 0
05:0028 0x63668b760388 ← 0
06:0030 0x63668b760390 ← 0
07:0038 0x63668b760398 ← 0
08:0040 0x63668b7603a0 ← 0
09:0048 0x63668b7603a8 ← 0
0a:0050 0x63668b7603b0 ← 0
0b:0058 0x63668b7603b8 ← 0
0c:0060 0x63668b7603c0 ← 0
0d:0068 0x63668b7603c8 ← 0
0e:0070 0x63668b7603d0 ← 0
0f:0078 0x63668b7603d8 ← 0
10:0080 0x63668b7603e0 ← 0
11:0088 0x63668b7603e8 ← 0
12:0090 0x63668b7603f0 ← 0
13:0098 0x63668b7603f8 ← 0
14:00a0 0x63668b760400 ← 0
15:00a8 0x63668b760408 ← 0
16:00b0 0x63668b760410 ← 0
17:00b8 0x63668b760418 ← 0
18:00c0 0x63668b760420 ← 0x90
19:00c8 0x63668b760428 → 0x63668b760bd0 ← 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaa'
1a:00d0 0x63668b760430 ← 0
1b:00d8 0x63668b760438 ← 0
1c:00e0 0x63668b760440 ← 0
1d:00e8 0x63668b760448 ← 0

```

所以通过堆风水，构造一下unsorted bin chunk，就能将libc段的地址放进这段堆空间

```

0a:0050 0x63668b760920 ← 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
0b:0058 0x63668b760928 ← 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
0c:0060 0x63668b760930 ← 'aaaaaaaaaaaaaaaaaaaaa'
0d:0068 0x63668b760938 ← 'aaaaaaa'
0e:0070 0x63668b760940 ← 0
0f:0078 0x63668b760948 ← 0x281
10:0080 0x63668b760950 → 0x7d3e36cf1b20 (main_arena+96) → 0x63668b760c60 ← 0
11:0088 0x63668b760958 → 0x7d3e36cf1b20 (main_arena+96) → 0x63668b760c60 ← 0
12:0090 0x63668b760960 ← 0x6161616161616161 ('aaaaaaa')
13:0098 0x63668b760968 ← 0x6161616161616161 ('aaaaaaa')
14:00a0 0x63668b760970 ← 0x6161616161616161 ('aaaaaaa')
15:00a8 0x63668b760978 ← 0x6161616161616161 ('aaaaaaa')
16:00b0 0x63668b760980 ← 0x6161616161616161 ('aaaaaaa')
17:00b8 0x63668b760988 ← 0x6161616161616161 ('aaaaaaa')

```

但是记得处理一下%s的截断问题，后续通过该利用链可以一直泄露出栈基址，堆基址以及canary。

自定义包的发送

稍微一读其实就会发现我们有一个隐藏的upload选项,

```
pkt.title[strcspn(pkt.title, "\n")] = 0;
```

这行代码本意是将换行符替换成\x00，来给字符串添加一个结束符，但是，假如前面的read恰好读了32字节，那么就会出现一个问题，strcspn函数会一直向下读直到遇到一个\n，也就是0x0a，因此，我们能够通过这个漏洞去改写掉当前包的长度，通过构造长度的错位，这时候，一个本来很长的包，被解析器解析了一段就停止了，那么剩下的内容就会被当成一个新的包来进行解析，此时就实现了自定义包的发送，我们可以自己决定包的标志位字段，从而多了很多利用的手段。

控制流的劫持

为了降低该题的难度，我还送了一个win函数，因此后续只要将控制流劫持到这个函数上，就可以将flag读取出来，而最后一个漏洞存在于bug处理函数中，存在一个栈溢出，结合前面的泄露，就能够完成控制流的劫持。